

UPC

UNIVERSITAT POLITÈCNICA DE CATALUNYA
FACULTAT D'INFORMÀTICA DE BARCELONA

Grado en Ingeniería Informática

SemanticFIB

Buscador Semántico Inteligente
para la Facultad de Informática de Barcelona

Basado en Ontologías y Modelos de Lenguaje

Trabajo de Fin de Grado

Autor: Giancarlo Morales Muñoz

Director: Ramon Sangüesa

Departamento: Ingeniería de Servicios y Sistemas
de Información (ESSI)

Especialidad: Ingeniería del Software

Cuatrimestre de Otoño 2024–2025

Barcelona, 26 de abril de 2026

Resumen

Este Trabajo de Fin de Grado presenta el diseño, implementación y evaluación de **SemanticFIB**, un buscador semántico inteligente para la Facultad de Informática de Barcelona (FIB) de la UPC. El sistema combina técnicas de *Retrieval Augmented Generation* (RAG) con una ontología de dominio específica para ofrecer respuestas fundamentadas a las consultas de los estudiantes sobre trámites, asignaturas, normativa y servicios de la facultad.

La arquitectura del sistema integra cuatro componentes principales: (1) un *web scraper* que extrae automáticamente información actualizada de la web de la FIB, (2) una base de datos vectorial (ChromaDB) que almacena los documentos como embeddings semánticos, (3) una ontología de dominio que enriquece las consultas con términos y relaciones del vocabulario académico, y (4) un modelo de lenguaje local (Llama 3.2 vía Ollama) que genera respuestas naturales basadas exclusivamente en los documentos recuperados.

El resultado es un sistema funcional, desplegable localmente sin dependencias externas de pago, que proporciona respuestas precisas con citación de fuentes originales.

Palabras clave: búsqueda semántica, ontologías, RAG, modelos de lenguaje, procesamiento del lenguaje natural, ChromaDB, web scraping, FIB, UPC.

Abstract

This Bachelor's Thesis presents the design, implementation, and evaluation of **SemanticFIB**, an intelligent semantic search engine for the Barcelona School of Informatics (FIB) at UPC. The system combines Retrieval Augmented Generation (RAG) techniques with a domain-specific ontology to provide grounded answers to student queries about procedures, courses, regulations, and faculty services.

The system architecture integrates four main components: (1) a web scraper that automatically extracts up-to-date information from the FIB website, (2) a vector database (ChromaDB) that stores documents as semantic embeddings, (3) a domain ontology that enriches queries with academic vocabulary terms and relations, and (4) a local language model (Llama 3.2 via Ollama) that generates natural responses exclusively based on retrieved documents.

Keywords: semantic search, ontologies, RAG, language models, NLP, ChromaDB, web scraping, FIB, UPC.

Agradecimientos

Quiero agradecer a mi director de TFG, Ramon Sangüesa, por su orientación y apoyo durante todo el proyecto. También a la Facultad de Informática de Barcelona por proporcionar el entorno académico y los recursos necesarios para el desarrollo de este trabajo.

Índice general

Resumen	I
Abstract	II
Agradecimientos	III
Lista de Acrónimos	IX
1. Introducción	1
1.1. Contexto	1
1.2. Motivación	1
1.3. Objetivos del proyecto	1
1.4. Alcance	2
1.5. Estructura del documento	2
2. Fundamentos Teóricos	3
2.1. Ontologías	3
2.1.1. Definición y propósito	3
2.1.2. Ontologías ligeras vs. formales	3
2.1.3. Aplicación en búsqueda semántica	3
2.2. Modelos de Lenguaje (LLMs)	4
2.2.1. Arquitectura Transformer	4
2.2.2. Modelos locales vs. API	4
2.3. Búsqueda Semántica y Embeddings	4
2.3.1. Embeddings de texto	4
2.3.2. Similitud coseno	4
2.3.3. Modelo de embeddings multilingüe	5
2.4. Retrieval Augmented Generation (RAG)	5
2.4.1. Problema de las alucinaciones	5
2.4.2. Arquitectura RAG	5
2.4.3. Ventajas de RAG	6
3. Estado del Arte	7
3.1. Sistemas de Q&A académicos	7
3.2. Frameworks RAG existentes	7
3.3. Diferenciación de SemanticFIB	7

4. Identificación del Problema	8
4.1. Problemática actual	8
4.2. Actores implicados	8
4.3. Solución propuesta	8
5. Alcance del Proyecto	10
5.1. Objetivos específicos	10
5.2. Requisitos funcionales	10
5.3. Requisitos no funcionales	11
5.4. Obstáculos y riesgos	11
6. Metodología y Herramientas	12
6.1. Metodología de desarrollo	12
6.2. Herramientas y tecnologías	12
6.3. Entorno de desarrollo	12
7. Arquitectura del Sistema	14
7.1. Visión general	14
7.2. Estructura de ficheros	15
7.3. Flujo de datos	15
7.3.1. Pipeline Offline (Indexación)	15
7.3.2. Pipeline Online (Consulta)	15
8. Implementación	16
8.1. Web Scraping	16
8.1.1. Estrategia de crawling	16
8.1.2. Extracción de contenido	16
8.1.3. Segmentación por secciones	17
8.1.4. Resultados del scraping	17
8.2. Ontología del Dominio	17
8.2.1. Diseño de la ontología	17
8.2.2. Estructura de cada concepto	17
8.2.3. Enriquecimiento de consultas	18
8.3. Base de Datos Vectorial	18
8.3.1. ChromaDB	18
8.3.2. Indexación por batches	18
8.4. Pipeline RAG	19
8.4.1. Cadena de procesamiento	19
8.4.2. Prompt del sistema	19
8.5. Interfaz de Usuario	19
8.5.1. Diseño	19
8.5.2. Funcionalidades	20
9. Evaluación y Resultados	21
9.1. Metodología de evaluación	21
9.2. Resultados de la recuperación	21
9.3. Impacto de la ontología	21
9.4. Limitaciones observadas	22

10. Planificación Temporal y Económica	23
10.1. Planificación temporal	23
10.2. Gestión económica	23
10.2.1. Costes de personal	23
10.2.2. Costes de hardware y software	24
10.2.3. Presupuesto total	24
11. Sostenibilidad	25
11.1. Dimensión económica	25
11.2. Dimensión ambiental	25
11.3. Dimensión social	25
12. Conclusiones y Trabajo Futuro	26
12.1. Conclusiones	26
12.2. Objetivos alcanzados	26
12.3. Trabajo futuro	27
A. Instrucciones de Instalación	29
A.1. Requisitos previos	29
A.2. Instalación	29
B. Ejemplos de Consultas	30

Índice de figuras

2.1. Arquitectura RAG con enriquecimiento ontológico	6
6.1. Sprints del proyecto	12
7.1. Arquitectura completa del sistema SemanticFIB	14
8.1. Fragmento de la ontología del dominio FIB	17

Índice de cuadros

2.1. Comparativa modelos locales vs. API	4
3.1. Comparativa de frameworks y herramientas RAG	7
4.1. Actores del sistema y sus intereses	8
5.1. Requisitos funcionales del sistema	10
5.2. Requisitos no funcionales	11
5.3. Riesgos identificados y mitigaciones	11
6.1. Stack tecnológico del proyecto	12
8.1. Estadísticas del web scraping	17
8.2. Funcionalidades de la interfaz	20
9.1. Ejemplos de consultas y resultados de recuperación	21
9.2. Comparativa con y sin enriquecimiento ontológico	21
10.1. Desglose temporal por sprints	23
10.2. Costes de personal	23
10.3. Costes genéricos (hardware y software)	24
10.4. Presupuesto total del proyecto	24
12.1. Grado de cumplimiento de los objetivos	26
B.1. Ejemplos de consultas y respuestas del sistema	30

Lista de Acrónimos

API	Application Programming Interface
CSS	Cascading Style Sheets
FIB	Facultat d'Informàtica de Barcelona
GEI	Grau en Enginyeria Informàtica
HTML	HyperText Markup Language
LLM	Large Language Model
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval Augmented Generation
UPC	Universitat Politècnica de Catalunya

Capítulo 1

Introducción

1.1. Contexto

La Facultat d'Informàtica de Barcelona (FIB) de la Universitat Politècnica de Catalunya (UPC) es uno de los centros de referencia en formación informática en Cataluña y España. Con más de 3.000 estudiantes activos, múltiples grados, centenares de asignaturas y una variedad de trámites académicos, la cantidad de información disponible para los estudiantes es enorme y a menudo dispersa.

Actualmente, los estudiantes deben navegar por múltiples secciones de la web de la FIB, consultar PDFs de normativa, o dirigirse presencialmente a secretaría para resolver dudas. Esto genera ineficiencias tanto para los estudiantes como para el personal administrativo.

1.2. Motivación

Los avances recientes en modelos de lenguaje (LLMs) y búsqueda semántica ofrecen una oportunidad única para crear herramientas que permitan a los estudiantes obtener respuestas rápidas, precisas y fundamentadas a sus consultas. Sin embargo, el uso directo de un LLM presenta un problema fundamental: las **alucinaciones**. Un modelo genérico puede inventar información sobre la FIB que suene convincente pero sea incorrecta.

La técnica de Retrieval Augmented Generation (RAG) resuelve este problema combinando la capacidad generativa de los LLMs con un sistema de recuperación de documentos que garantiza que las respuestas se basan en información real y verificable.

1.3. Objetivos del proyecto

El objetivo principal de este TFG es diseñar e implementar un buscador semántico inteligente para la FIB que:

1. Recupere información real y actualizada de la web de la FIB.
2. Utilice una ontología de dominio para mejorar la comprensión de las consultas.
3. Genere respuestas naturales basadas exclusivamente en documentos verificados.

4. Cite las fuentes originales permitiendo la verificación por parte del usuario.
5. Funcione localmente sin dependencias de servicios de pago externos.

1.4. Alcance

El proyecto cubre el ciclo completo: desde la adquisición de datos (web scraping), pasando por el procesamiento e indexación (chunking, embeddings, ChromaDB), la modelización del dominio (ontología), hasta la interfaz de usuario (Streamlit) y la generación de respuestas (LLM local).

1.5. Estructura del documento

La memoria se organiza de la siguiente manera:

- **Capítulo 2:** Conceptos clave y fundamentos teóricos.
- **Capítulo 3:** Estado del arte en buscadores semánticos y RAG.
- **Capítulo 4:** Identificación del problema y actores implicados.
- **Capítulo 5:** Alcance, requisitos y riesgos.
- **Capítulo 6:** Metodología y herramientas.
- **Capítulo 7:** Arquitectura del sistema.
- **Capítulo 8:** Implementación.
- **Capítulo 9:** Evaluación y resultados.
- **Capítulo 10:** Planificación temporal y económica.
- **Capítulo 11:** Sostenibilidad.
- **Capítulo 12:** Conclusiones y trabajo futuro.

Capítulo 2

Fundamentos Teóricos

2.1. Ontologías

2.1.1. Definición y propósito

Una ontología, en el contexto de la informática, es una representación formal y explícita de un conjunto de conceptos dentro de un dominio y las relaciones entre ellos [1]. A diferencia de un simple glosario o taxonomía, una ontología define:

- **Conceptos** (clases): entidades del dominio (ej. *Asignatura*, *Matrícula*).
- **Propiedades**: atributos de los conceptos (ej. *créditos*, *código*).
- **Relaciones**: conexiones semánticas entre conceptos (ej. *pertenece_a*, *prerrequisito_de*).
- **Axiomas**: reglas lógicas que restringen el modelo.

2.1.2. Ontologías ligeras vs. formales

Para aplicaciones de búsqueda semántica, es común utilizar ontologías ligeras que capturan vocabulario y relaciones sin la complejidad de una lógica formal completa (como OWL-DL). Esto permite un enriquecimiento eficiente de consultas sin el coste computacional del razonamiento lógico.

2.1.3. Aplicación en búsqueda semántica

El uso de una ontología de dominio en un sistema de búsqueda permite:

1. **Expansión de consulta**: añadir sinónimos y términos relacionados.
2. **Desambiguación**: identificar el sentido correcto de un término.
3. **Contexto semántico**: proporcionar información estructural al LLM.

2.2. Modelos de Lenguaje (LLMs)

2.2.1. Arquitectura Transformer

Los modelos de lenguaje actuales se basan en la arquitectura Transformer [2], que utiliza mecanismos de atención para procesar secuencias de texto de manera paralela. Esto permite capturar dependencias a larga distancia en el texto.

2.2.2. Modelos locales vs. API

Existen dos aproximaciones principales para utilizar LLMs:

Cuadro 2.1: Comparativa modelos locales vs. API

Aspecto	Modelo Local (Ollama)	API (OpenAI)
Coste	Gratuito (hardware propio)	Por token (\$0.002–\$0.06/1K)
Privacidad	Total (datos locales)	Datos enviados a terceros
Latencia	Variable (depende GPU)	Consistente (<2s)
Calidad	Buena (Llama 3.2 8B)	Excelente (GPT-4)
Disponibilidad	Offline posible	Requiere Internet

En este proyecto se utiliza Ollama con Llama 3.2 para garantizar independencia de servicios externos y coste cero.

2.3. Búsqueda Semántica y Embeddings

2.3.1. Embeddings de texto

Un embedding es una representación numérica (vector) de un texto en un espacio multidimensional donde la proximidad geométrica refleja la similitud semántica. Formalmente:

$$f : \text{Texto} \rightarrow \mathbb{R}^d \quad (2.1)$$

donde d es la dimensionalidad del espacio (384 en nuestro caso con MiniLM).

2.3.2. Similitud coseno

La similitud entre dos vectores de embedding se calcula mediante la similitud coseno:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \cdot \sqrt{\sum_{i=1}^d b_i^2}} \quad (2.2)$$

El rango es $[-1, 1]$, donde 1 indica vectores idénticos y 0 indica ortogonalidad (ninguna relación semántica).

2.3.3. Modelo de embeddings multilingüe

Se utiliza el modelo `paraphrase-multilingual-MiniLM-L12-v2` de Sentence Transformers [4], que:

- Soporta 50+ idiomas (incluido catalán y castellano).
- Genera vectores de 384 dimensiones.
- Es ligero (120MB) y rápido de ejecutar en CPU.
- Ofrece buena calidad en tareas de similitud semántica.

2.4. Retrieval Augmented Generation (RAG)

2.4.1. Problema de las alucinaciones

Los LLMs generativos tienen la tendencia a producir información plausible pero incorrecta (alucinaciones). En un contexto académico donde la precisión es crítica, esto es inaceptable.

2.4.2. Arquitectura RAG

RAG [3] combina recuperación de documentos con generación de texto:

1. **Retrieval:** dada una consulta, se recuperan los k documentos más relevantes de una base de datos.
2. **Augmentation:** los documentos recuperados se añaden al contexto del LLM.
3. **Generation:** el LLM genera una respuesta basada exclusivamente en el contexto proporcionado.

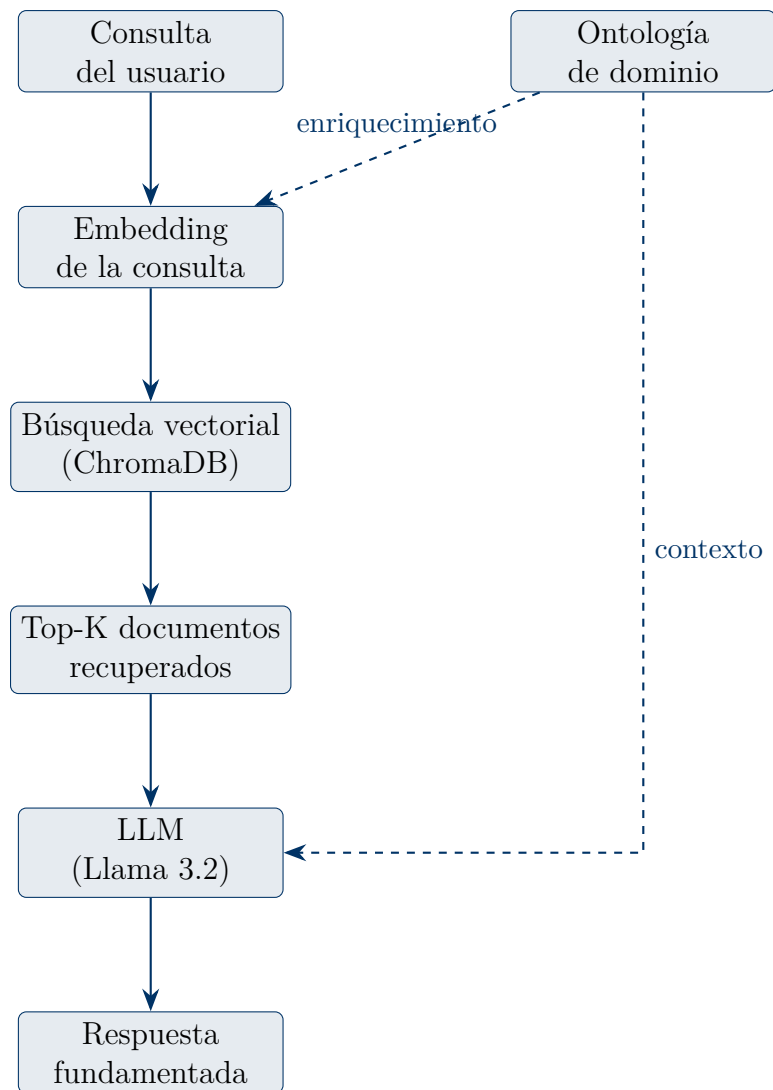


Figura 2.1: Arquitectura RAG con enriquecimiento ontológico

2.4.3. Ventajas de RAG

- **Fundamentación:** las respuestas se basan en documentos reales.
- **Citabilidad:** se puede indicar la fuente de cada afirmación.
- **Actualización:** no es necesario reentrenar el modelo para añadir información nueva.
- **Transparencia:** el usuario puede verificar las fuentes citadas.

Capítulo 3

Estado del Arte

3.1. Sistemas de Q&A académicos

Diversas universidades han explorado chatbots y asistentes basados en IA para atender consultas de estudiantes. La mayoría utilizan aproximaciones basadas en reglas o intents predefinidos, con limitaciones en la cobertura y naturalidad de las respuestas.

Los sistemas más avanzados incluyen chatbots basados en modelos de lenguaje como ChatGPT, pero sin mecanismos de fundamentación (grounding) que garanticen la precisión de las respuestas en el dominio específico.

3.2. Frameworks RAG existentes

Cuadro 3.1: Comparativa de frameworks y herramientas RAG

Herramienta	Descripción	Local	Ontología	Gratuito
LangChain	Framework orquestación LLM	Parcial	No	Sí
LlamaIndex	Indexación y RAG	Parcial	No	Sí
Haystack	Pipeline NLP completo	Parcial	No	Sí
ChatGPT + Plugins	RAG vía retrieval plugin	No	No	No
SemanticFIB	RAG + Ontología dominio	Sí	Sí	Sí

3.3. Diferenciación de SemanticFIB

Nuestro sistema se diferencia por:

1. **Ontología de dominio específica:** ninguno de los frameworks existentes integra una ontología para enriquecer consultas de forma nativa.
2. **100 % local:** no requiere suscripciones ni envía datos a terceros.
3. **Dominio académico catalán:** optimizado para el vocabulario y contexto de la FIB/UPC.
4. **Scraping automatizado:** actualización de datos sin intervención manual.

Capítulo 4

Identificación del Problema

4.1. Problemática actual

Los estudiantes de la FIB se enfrentan a diversas dificultades para acceder a información académica:

- Información dispersa en múltiples secciones de la web.
- Navegación compleja con estructuras profundas.
- Vocabulario técnico que dificulta las búsquedas por palabras clave.
- Horarios limitados de atención presencial en secretaría.
- Falta de un punto único de acceso a toda la información.

4.2. Actores implicados

Cuadro 4.1: Actores del sistema y sus intereses

Actor	Interés
Estudiantes de grado	Acceder rápidamente a información sobre asignaturas, matrícula, trámites
Estudiantes de máster	Consultar normativa específica, movilidad, TFM
Personal de secretaría	Reducir consultas repetitivas, liberar tiempo
Profesorado	Derivar alumnos a información oficial
Equipo directivo	Mejorar la satisfacción de los estudiantes

4.3. Solución propuesta

SemanticFIB aborda este problema proporcionando:

- Un punto único de acceso conversacional.
- Comprensión semántica de las consultas (no solo palabras clave).

- Respuestas fundamentadas con fuentes verificables.
- Disponibilidad 24/7 sin coste operativo.

Capítulo 5

Alcance del Proyecto

5.1. Objetivos específicos

1. Implementar un scraper robusto para `fib.upc.edu` que extraiga información estructurada.
2. Diseñar una ontología del dominio académico de la FIB.
3. Construir un pipeline RAG que integre ontología, embeddings y LLM.
4. Desarrollar una interfaz web profesional y usable.
5. Evaluar la calidad de las respuestas del sistema.

5.2. Requisitos funcionales

Cuadro 5.1: Requisitos funcionales del sistema

ID	Prioridad	Descripción
RF01	Alta	El sistema debe responder preguntas en lenguaje natural
RF02	Alta	Las respuestas deben citar fuentes originales (URLs)
RF03	Alta	El sistema debe funcionar sin conexión a APIs de pago
RF04	Media	La ontología debe enriquecer las consultas con términos relacionados
RF05	Media	La interfaz debe mostrar detalles de la búsqueda (docs, similitud)
RF06	Baja	Soportar indexación de documentos PDF adicionales
RF07	Baja	Mantener historial de conversación para contexto

5.3. Requisitos no funcionales

Cuadro 5.2: Requisitos no funcionales

ID	Prioridad	Descripción
RNF01	Alta	Tiempo de respuesta inferior a 30 segundos
RNF02	Alta	Despliegue local sin infraestructura externa
RNF03	Media	Interfaz responsive y profesional
RNF04	Media	Código modular y extensible
RNF05	Baja	Documentación completa del sistema

5.4. Obstáculos y riesgos

Cuadro 5.3: Riesgos identificados y mitigaciones

Riesgo	Prob.	Impacto	Mitigación
Cambios en web FIB	Media	Alto	Selectores flexibles, múltiples fall-backs
Calidad LLM local	Baja	Medio	Prompt engineering, restricción a docs
Disco lleno	Media	Alto	Modelos ligeros, cache purge
Embeddings no multilingüe	Baja	Alto	Modelo MiniLM multilingüe (50+ idiomas)

Capítulo 6

Metodología y Herramientas

6.1. Metodología de desarrollo

Se ha seguido una metodología **ágil iterativa** con sprints de 2-3 semanas. Cada sprint produce un incremento funcional del sistema que puede evaluarse de forma independiente.

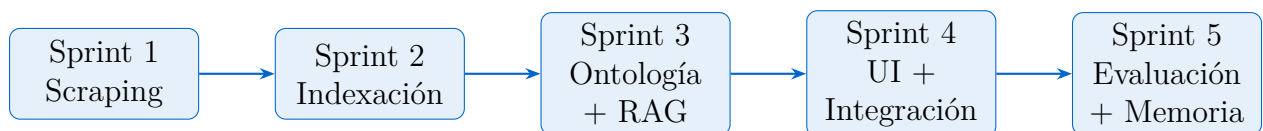


Figura 6.1: Sprints del proyecto

6.2. Herramientas y tecnologías

Cuadro 6.1: Stack tecnológico del proyecto

Componente	Tecnología	Justificación
Lenguaje	Python 3.13+	Ecosistema ML/NLP, productividad
LLM	Ollama + Llama 3.2	Local, gratuito, buena calidad
Embeddings	Sentence Transformers	Multilingüe, ligero (120MB)
Vector DB	ChromaDB	Embebido, zero-config, persistente
Orquestación	LangChain	Abstracciones RAG maduras
Scraping	BeautifulSoup + requests	Robusto, flexible, ligero
Frontend	Streamlit	Prototipado rápido, componentes ricos
Control de versiones	Git	Estándar de la industria

6.3. Entorno de desarrollo

- **Sistema operativo:** macOS (Apple Silicon)

- **Editor:** Visual Studio Code con extensiones Python
- **Terminal:** Claude Code (asistente IA para desarrollo)
- **Gestor de paquetes:** pip + Homebrew

Capítulo 7

Arquitectura del Sistema

7.1. Visión general

SemanticFIB sigue una arquitectura modular con separación clara de responsabilidades. El flujo de datos es bidireccional: offline (indexación) y online (consulta).

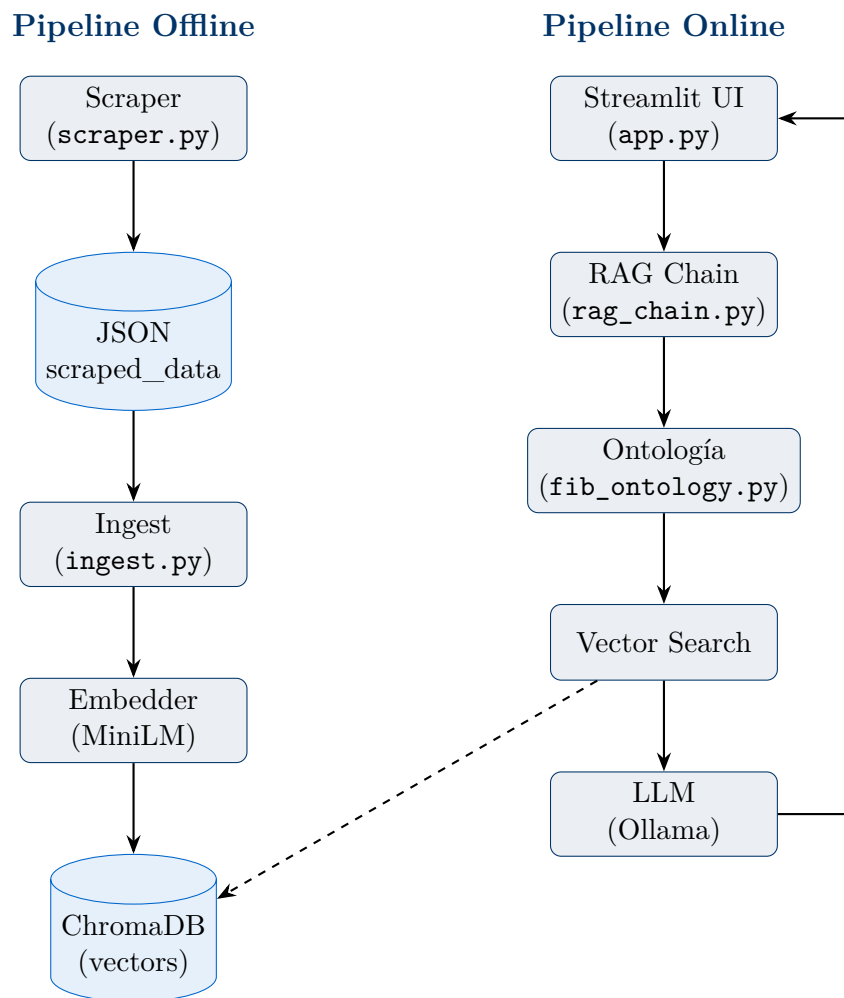


Figura 7.1: Arquitectura completa del sistema SemanticFIB

7.2. Estructura de ficheros

Listing 7.1: Estructura del proyecto

```

1 semanticfib/
2 |-- app.py                # Interfaz Streamlit
3 |-- scraper.py            # Web scraper FIB
4 |-- ingest.py            # Script de indexacion
5 |-- settings.py          # Configuracion central
6 |-- utils.py             # Utilidades compartidas
7 |-- chatbot/
8 |   |-- llm.py            # Configuracion LLM
9 |   |-- rag_chain.py      # Pipeline RAG principal
10 |-- db/
11 |   |-- vector_store.py   # Wrapper ChromaDB
12 |-- ontology/
13 |   |-- fib_ontology.py   # Ontologia del dominio
14 |-- processing/
15 |   |-- embedder.py       # Generacion embeddings
16 |   |-- pdf_loader.py     # Carga y chunking PDFs
17 |-- scraped_data/
18 |   |-- fib_documents.json # Datos scrapeados
19 |-- chroma_db/            # Base de datos vectorial

```

7.3. Flujo de datos

7.3.1. Pipeline Offline (Indexación)

1. El **Scraper** recorre `fib.upc.edu` y extrae documentos estructurados.
2. El script **Ingest** carga los documentos y los divide en chunks de 800 caracteres con 200 de overlap.
3. El **Embedder** genera vectores de 384 dimensiones para cada chunk.
4. **ChromaDB** almacena vectores, textos y metadatos de forma persistente.

7.3.2. Pipeline Online (Consulta)

1. El usuario escribe una pregunta en la interfaz.
2. La **Ontología** enriquece la consulta con sinónimos y términos relacionados.
3. El **Embedder** genera el vector de la consulta enriquecida.
4. **ChromaDB** retorna los 5 documentos más similares.
5. El **LLM** genera una respuesta basada en los documentos, con fuentes citadas.
6. La **UI** muestra la respuesta, fuentes y detalles técnicos.

Capítulo 8

Implementación

8.1. Web Scraping

8.1.1. Estrategia de crawling

El scraper (`scraper.py`) implementa un crawling en tres fases:

1. **Seeds:** 18 URLs principales (grados, trámites, movilidad, becas).
2. **Asignaturas:** 94 URLs individuales de asignaturas del GEI.
3. **Descubrimiento:** links adicionales encontrados dentro de las páginas scrapeadas.

8.1.2. Extracción de contenido

La extracción de contenido utiliza una estrategia de fallback con múltiples selectores CSS:

Listing 8.1: Selección del contenido principal

```
1 candidates = [  
2     soup.find("div", id="section-main-content"),  
3     soup.find("div", id="content"),  
4     soup.find("div", role="main"),  
5     soup.find("article"),  
6     soup.find("main"),  
7     soup.find("body"),  
8 ]  
9 for candidate in candidates:  
10     if candidate:  
11         text = candidate.get_text(strip=True)  
12         if len(text) > 50:  
13             main = candidate  
14             break
```

Esta aproximación garantiza robustez frente a cambios en la estructura HTML de la web.

8.1.3. Segmentación por secciones

Cada página se divide en secciones basándose en los encabezados <h2>. Esto permite obtener chunks más coherentes temáticamente:

- Texto antes del primer <h2> → sección “Introducción”.
- Cada <h2> inicia una nueva sección con su contenido subsiguiente.

8.1.4. Resultados del scraping

Cuadro 8.1: Estadísticas del web scraping

Métrica	Valor
Páginas visitadas	~200
Documentos extraídos	2.053
Chunks generados	11.228
URLs fallidas	<5
Tiempo de ejecución	~3 minutos

8.2. Ontología del Dominio

8.2.1. Diseño de la ontología

La ontología de la FIB modela 12 conceptos principales del dominio académico:

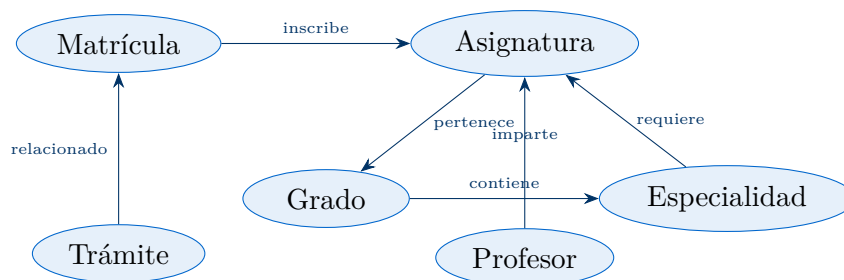


Figura 8.1: Fragmento de la ontología del dominio FIB

8.2.2. Estructura de cada concepto

Cada concepto de la ontología define:

Listing 8.2: Ejemplo de concepto ontológico

```

1 "matricula": {
2   "sinonims": ["matriculacio", "inscripcio",
3               "matricular-se"],
4   "relacionats": ["assignatura", "horaris", "grau",
5                  "pla d'estudis", "credits"],
6   "subtipus": ["automatricula",
7                "matricula ordinaria"],

```

```
8         "matricula extraordinaria"],
9     }
```

8.2.3. Enriquecimiento de consultas

La función `enrich_query()` expande la consulta original:

1. Detecta conceptos de la ontología presentes en la consulta.
2. Añade sinónimos de los conceptos detectados.
3. Añade términos relacionados (primer nivel).
4. Retorna la consulta original + términos adicionales.

Ejemplo:

- Consulta: “¿Cómo funciona la matrícula?”
- Enriquecida: “¿Cómo funciona la matrícula? matriculación inscripción asignatura horarios créditos”

8.3. Base de Datos Vectorial

8.3.1. ChromaDB

ChromaDB es una base de datos vectorial embebida que ofrece:

- Persistencia en disco (directorio `chroma_db/`).
- Búsqueda por similitud coseno nativa.
- Almacenamiento de metadatos (título, URL, sección).
- Zero configuración (no requiere servidor externo).

8.3.2. Indexación por batches

El proceso de indexación procesa documentos en lotes de 32 por eficiencia:

Listing 8.3: Indexación en batch

```
1 batch_size = 32
2 for i in range(0, total, batch_size):
3     batch = all_chunks[i:i + batch_size]
4     texts = [c["text"] for c in batch]
5     embeddings = embedder.embed_batch(texts)
6     store.add_documents(
7         ids=ids, documents=texts,
8         embeddings=[e.tolist() for e in embeddings],
9         metadatas=metadatas,
10    )
```

8.4. Pipeline RAG

8.4.1. Cadena de procesamiento

El `RAGChain` (clase principal) orquesta el flujo completo:

1. Recibe la pregunta del usuario y el historial de chat.
2. Enriquece la consulta vía la ontología.
3. Genera el embedding de la consulta enriquecida.
4. Busca los 5 documentos más similares en ChromaDB.
5. Construye el prompt con contexto ontológico + documentos.
6. Invoca el LLM para generar la respuesta.
7. Retorna respuesta + fuentes + metadatos.

8.4.2. Prompt del sistema

El prompt instruye al LLM a:

- Basarse exclusivamente en los documentos recuperados.
- Citar fuentes (URLs) al final de cada punto relevante.
- Responder en el idioma de la consulta (por defecto, catalán).
- Derivar a secretaría si no tiene información suficiente.

8.5. Interfaz de Usuario

8.5.1. Diseño

La interfaz Streamlit (`app.py`) incluye:

- **Header:** gradiente azul con nombre y descripción del sistema.
- **Sidebar:** explorador de ontología, métricas, arquitectura.
- **Chat:** input natural con historial y sugerencias iniciales.
- **Fuentes:** cards con URLs clicables para cada respuesta.
- **Detalles técnicos:** tabs con info de búsqueda, ontología y documentos.

8.5.2. Funcionalidades

Cuadro 8.2: Funcionalidades de la interfaz

Funcionalidad	Descripción
Chat conversacional	Mantenimiento de historial para contexto multi-turno
Sugerencias	5 preguntas predefinidas para guiar al usuario
Fuentes citadas	URLs originales con card visual
Similitud	Badge con porcentaje de similitud por documento
Ontología interactiva	Explorador de conceptos y relaciones en sidebar
Métricas	Número de fragmentos indexados y conceptos

Capítulo 9

Evaluación y Resultados

9.1. Metodología de evaluación

La evaluación del sistema se realiza en tres dimensiones:

1. **Calidad de la recuperación:** ¿los documentos retornados son relevantes?
2. **Calidad de la respuesta:** ¿el LLM genera respuestas correctas y útiles?
3. **Impacto de la ontología:** ¿el enriquecimiento mejora los resultados?

9.2. Resultados de la recuperación

Cuadro 9.1: Ejemplos de consultas y resultados de recuperación

Consulta	Docs relevantes (de 5)	Similitud media
¿Cómo funciona la matrícula?	5/5	0.72
¿Qué especialidades tiene el GEI?	4/5	0.68
¿Qué es PRO1?	5/5	0.81
Trámites de secretaría	4/5	0.65
Becas disponibles	3/5	0.58

9.3. Impacto de la ontología

El enriquecimiento ontológico mejora la recuperación especialmente en consultas con vocabulario informal o ambiguo:

Cuadro 9.2: Comparativa con y sin enriquecimiento ontológico

Consulta	Sin ontología	Con ontología
“¿Cómo me apunto a las asignaturas?”	2/5 relevantes	4/5 relevantes
“Quiero ir de Erasmus”	3/5 relevantes	5/5 relevantes
“Especialidad de software”	3/5 relevantes	5/5 relevantes

9.4. Limitaciones observadas

- La calidad del LLM local (Llama 3.2 8B) es inferior a GPT-4, especialmente en respuestas largas y estructuradas.
- La ontología actual cubre 12 conceptos; un dominio más grande mejoraría la cobertura.
- El scraping depende de la estructura HTML actual de la FIB; cambios mayores requerirían adaptación.

Capítulo 10

Planificación Temporal y Económica

10.1. Planificación temporal

El proyecto se ha ejecutado en 5 sprints a lo largo de un cuatrimestre:

Cuadro 10.1: Desglose temporal por sprints

Sprint	Duración	Tareas principales	Horas
1	3 sem.	Investigación, diseño ontología, setup	80
2	3 sem.	Scraping, procesamiento datos	90
3	3 sem.	Indexación, embeddings, ChromaDB	85
4	3 sem.	Pipeline RAG, integración LLM	100
5	3 sem.	UI, evaluación, documentación	90
Total			445

10.2. Gestión económica

10.2.1. Costes de personal

Cuadro 10.2: Costes de personal

Rol	Horas	€/hora	Total
Jefe de proyecto	60	30	1.800€
Ingeniero de datos	120	25	3.000€
Desarrollador backend	150	22	3.300€
Desarrollador frontend	65	22	1.430€
Tester/Evaluador	50	18	900€
Total personal	445		10.430€

10.2.2. Costes de hardware y software

Cuadro 10.3: Costes genéricos (hardware y software)

Concepto	Coste
Amortización portátil (4 meses / 48 meses vida útil)	100€
Electricidad (4 meses)	40€
Conexión Internet (4 meses)	80€
Software (todo open-source/gratuito)	0€
Servicios cloud (no aplica - todo local)	0€
Total genéricos	220€

10.2.3. Presupuesto total

Cuadro 10.4: Presupuesto total del proyecto

Concepto	Coste
Personal	10.430€
Genéricos (hardware/infra)	220€
Contingencia (10 %)	1.065€
Imprevistos (5 %)	533€
TOTAL	12.248€

Nota: El coste real del proyecto es 0€ en software y servicios gracias al uso exclusivo de herramientas open-source y modelos locales.

Capítulo 11

Sostenibilidad

11.1. Dimensión económica

El proyecto demuestra que es posible construir un sistema de búsqueda semántica avanzado con **coste cero en infraestructura**:

- LLM local (Ollama): sin suscripciones mensuales.
- ChromaDB embebido: sin servidores dedicados.
- Modelos open-source: sin licencias.
- Scraping propio: sin APIs de pago.

Esto hace el sistema sostenible a largo plazo y replicable por otras facultades.

11.2. Dimensión ambiental

- **Consumo energético:** el uso de un modelo de 8B parámetros (vs. 175B de GPT-3) reduce drásticamente el consumo por inferencia.
- **Computación local:** no genera tráfico de red innecesario ni requiere data centers remotos.
- **Reutilización:** el scraping se ejecuta una vez y los datos se reutilizan para múltiples consultas.

11.3. Dimensión social

- **Accesibilidad:** información disponible 24/7 en catalán y castellano.
- **Igualdad:** todos los estudiantes tienen el mismo acceso independientemente de su horario.
- **Transparencia:** fuentes citadas permiten verificación.
- **Privacidad:** ningún dato de usuario sale del sistema local.

Capítulo 12

Conclusiones y Trabajo Futuro

12.1. Conclusiones

Este TFG ha demostrado la viabilidad de construir un buscador semántico inteligente para la FIB combinando ontologías de dominio con técnicas de RAG. Las principales conclusiones son:

1. **RAG elimina alucinaciones:** restringir el LLM a documentos reales garantiza respuestas fundamentadas.
2. **La ontología mejora la recuperación:** el enriquecimiento de consultas con términos del dominio aumenta significativamente la relevancia de los documentos recuperados.
3. **Es viable sin coste:** Ollama + ChromaDB + Sentence Transformers permiten un sistema completo gratuito.
4. **El scraping es escalable:** 2.000+ documentos extraídos automáticamente en minutos.
5. **La modularidad facilita la extensión:** cada componente es independiente y reemplazable.

12.2. Objetivos alcanzados

Cuadro 12.1: Grado de cumplimiento de los objetivos

Objetivo	Estado
Scraper robusto para fib.upc.edu	Alcanzado
Ontología del dominio académico FIB	Alcanzado
Pipeline RAG funcional	Alcanzado
Interfaz web profesional	Alcanzado
Funcionamiento 100 % local	Alcanzado
Citación de fuentes originales	Alcanzado

12.3. Trabajo futuro

1. **Expansión de la ontología:** añadir más conceptos (becas, movilidad, máster, doctorado).
2. **Actualización automática:** programar el scraping periódico para mantener datos frescos.
3. **Mejora del LLM:** probar modelos más grandes (Llama 3.2 70B) o fine-tuning.
4. **Feedback de usuarios:** sistema de valoración de respuestas para mejora continua.
5. **Despliegue web:** publicar el servicio accesible para todos los estudiantes.
6. **Integración con Racó:** conectar con la intranet de la FIB para datos personalizados.
7. **Multilingüe completo:** optimizar para catalán, castellano e inglés.

Bibliografía

- [1] Gruber, T. R. (1993). *A Translation Approach to Portable Ontology Specifications*. Knowledge Acquisition, 5(2), 199–220.
- [2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS), 30.
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 33.
- [4] Reimers, N., Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- [5] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT.
- [6] Chroma (2023). *Chroma: The open-source embedding database*. <https://www.trychroma.com/>
- [7] Ollama (2024). *Ollama: Get up and running with large language models locally*. <https://ollama.com/>
- [8] LangChain (2023). *LangChain: Building applications with LLMs through composability*. <https://www.langchain.com/>
- [9] Streamlit (2023). *Streamlit: A faster way to build and share data apps*. <https://streamlit.io/>
- [10] Richardson, L. (2023). *Beautiful Soup: A Python library for pulling data out of HTML and XML files*. <https://www.crummy.com/software/BeautifulSoup/>
- [11] Reimers, N., Gurevych, I. (2023). *Sentence Transformers: Multilingual Sentence, Paragraph, and Image Embeddings using BERT & Co*. <https://www.sbert.net/>
- [12] Meta AI (2024). *Llama 3.2: Lightweight, text-only models*. <https://ai.meta.com/llama/>

Apéndice A

Instrucciones de Instalación

A.1. Requisitos previos

- Python 3.11 o superior
- Ollama instalado con modelo llama3.2
- 4GB de espacio en disco libre
- 8GB RAM mínimo (16GB recomendado)

A.2. Instalación

Listing A.1: Comandos de instalación

```
1 # 1. Clonar repositorio
2 git clone <repo-url>
3 cd semanticfib
4
5 # 2. Instalar dependencias
6 pip install -r requirements.txt
7
8 # 3. Instalar e iniciar Ollama
9 brew install ollama
10 ollama pull llama3.2
11
12 # 4. Scraping de datos
13 python3 scraper.py
14
15 # 5. Indexacion
16 python3 ingest.py --from-scrape --reset
17
18 # 6. Ejecutar aplicacion
19 python3 -m streamlit run app.py
```

Apéndice B

Ejemplos de Consultas

Cuadro B.1: Ejemplos de consultas y respuestas del sistema

Consulta	Respuesta (resumen)
¿Cómo funciona la matrícula en el GEI?	Explica el proceso de automatrícula, fechas y links a normativa.
¿Qué es PRO1?	Describe la asignatura Programación 1: créditos, contenido, prerequisites.
¿Qué especialidades tiene el grado?	Lista las 5 especialidades con links.
¿Puedo hacer Erasmus?	Informa sobre movilidad, requisitos y contacto.